

“Design and Development of a Public Transport Live Tracker”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

R. MANIKANTA : 2520090228

T. SIDDHARTH : 2520090187

P. SUVISHAL PATEL : 2520090195

Under the Esteemed Guidance of

Dr. M. Gayatri

Associate Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “**Design and Development of a Public Transport Live Tracker**” is a record of Bonafide work of **R. MANIKANTA - 2520090228**, **T. SIDDHARTH - 2520090187**, **P. SUVISHAL PATEL - 2520090195** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

R. MANIKANTA – 2520090228

T. SIDDHARTH – 2520090187

P. SUVISHAL PATEL – 2520090195

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled “**Design and Development of a Public Transport Live Tracker**” is a bonafide work done and submitted by **R. MANIKANTA - 2520090228, T. SIDDHARTH - 2520090187, P. SUVISHAL PATEL - 2520090195** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our guide **Dr. M. Gayatri**, Associate Professor, Department of Computer Science and Information Technology, whose tremendous support, encouragement and guidance cannot be expressed in words.

We express our gratitude to the Head of the Department of Computer Science and Information Technology for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and all those who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

R. MANIKANTA – 2520090228

T. SIDDHARTH – 2520090187

P. SUVISHAL PATEL – 2520090195

Abstract

Public Transport Live Tracker is a web-based system designed to help passengers track public buses and other public transport vehicles in real time. Passengers often face problems such as uncertain arrival times, long waiting periods, and difficulty knowing the current location of a vehicle. The proposed system addresses these issues by providing live vehicle locations, route information, estimated arrival times, and stop details through an easy-to-use interface.

The system collects and processes location updates from transport vehicles and stores relevant route, vehicle, schedule and user information in a MySQL database. A Node.js and Express.js backend handles real-time data processing and exposes REST APIs and WebSocket channels to the React.js frontend. Users can select a route or vehicle to view its current location and estimated arrival at selected stops. The system also maintains transport schedules and provides basic notifications and status updates.

The project aims to improve the public transport experience by reducing uncertainty and waiting time, while giving transport operators organized vehicle and route information. The proposed solution demonstrates the use of database systems, backend development, APIs, and real-time data handling in a practical public transportation application.

Index

S. No.	Chapters	Topics	Page.no
	Acknowledgement		
	Abstract		
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	7
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools	9
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Database 3.4 Version Control 3.5 APIs or External services	10
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer 4.3 Deployment architecture	11
5	Design	5.1 Use case and data flow 5.2 ER Diagram / Database schema	12
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic 6.3 API integration 6.4 Authentication & Authorization	13
7	Features	7.1 List of main features 7.2 How each feature works	14
8	Testing	8.1 Postman testing 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	15
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	15
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	16
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	17
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	17
13	References	Books, tutorials, APIs, documentation sites used	18
14	Appendices	Screenshots; code snippets; setup instructions; user manual; geo-tag photos; review forms	18

CHAPTER 1 - INTRODUCTION

1.1 Background of the Project

Public transportation is a vital part of daily life for millions of commuters, students and workers. Buses, in particular, remain the most affordable and widely available mode of transport in most cities. Yet the experience of using them has changed very little over the years: passengers still rely on printed timetables or notice boards at bus stops, and these seldom reflect what is actually happening on the road.

Traffic congestion, breakdowns, route diversions and weather conditions regularly cause buses to run ahead of or behind schedule. A passenger standing at a stop has no reliable way to know whether the bus is five minutes away or has already left. The result is wasted time, missed connections, and a general loss of trust in public transport.

Advances in GPS hardware, mobile networks and web technologies now make it practical to track vehicles and share their positions instantly. Low-cost GPS modules can report a vehicle's coordinates every few seconds, and modern backend frameworks can receive and broadcast those updates to thousands of connected users. This project, **Public Transport Live Tracker**, uses these technologies to give passengers a live view of public transport vehicles.

From an academic perspective, the project brings together several core areas of computer science:

- **Database Systems:** designing a normalized relational schema for users, vehicles, routes, stops, schedules and location history.
- **Backend Development:** building REST APIs and real-time services with Node.js and Express.js.
- **Distributed / Real-time Systems:** handling a continuous stream of location updates and pushing them to many clients using WebSockets.
- **Web Front-end Development:** an interactive map-based interface built with React.js, HTML, CSS and JavaScript.

1.2 Problem Statement

Passengers who depend on public buses do not have dependable, real-time information about where their vehicle is or when it will arrive. Fixed schedules cannot account for delays, and physical information boards are rarely updated. Passengers may wait for long periods without knowing the arrival time, and route and stop information is often hard to find.

Transport operators face a parallel problem. Vehicle, route and schedule data is frequently managed in separate systems or by hand, which leads to inaccurate information, poor coordination and slow responses to disruptions. Existing systems also rarely offer notifications or vehicle status updates.

The core problem is therefore the absence of a unified, real-time, database-backed system that connects vehicle location data with route, stop and schedule information and presents it to passengers in a simple web interface.

1.3 Scope of the Project

The scope of the project covers the design and development of a web application with the following boundaries:

- Collection of vehicle location updates (GPS coordinates with timestamps) through an API, with a simulator used for demonstration.
- Storage of users, vehicles, routes, stops, schedules and location history in a MySQL database.
- Display of live vehicle positions on a map, together with route and stop details.
- Calculation of estimated arrival time (ETA) at a selected stop.
- An admin interface for managing vehicles, routes, stops and schedules.
- Basic notifications and service status updates to passengers.
- Out of scope: online ticketing and payments, driver-side native mobile applications, and integration with physical GPS hardware in production.

1.4 Objectives of the Project

- To provide passengers with the live location of buses on an interactive map.
- To display route information, stop lists and scheduled timings in an easy-to-use interface.
- To estimate and show the arrival time of a bus at any selected stop.
- To store and manage vehicle, route, stop, schedule and user data in a well-designed relational database.
- To build a backend service that processes location updates in real time and exposes REST APIs and WebSocket events.
- To provide basic notifications and status updates about delays or service changes.
- To reduce passenger waiting time and uncertainty, and to help operators manage fleets more effectively.

1.5 Importance of the Application

Real-time tracking changes how people use public transport. Passengers can leave home only when the bus is close, plan connections with confidence, and avoid long, uncertain waits at the stop. This is especially valuable for students, daily office commuters, elderly passengers and people travelling at night.

For operators, the system provides organized data about vehicles and routes, makes delays visible immediately, and creates a historical record of location data that can later be used to improve schedules and routes. At a societal level, a more reliable public transport service encourages people to use buses instead of private vehicles, which helps reduce traffic congestion and air pollution.

Academically, the project is a practical demonstration of how database design, backend development, APIs and real-time data handling work together in a single, real-world system.

1.6 Target Users / Audience

- **Passengers / commuters:** students, employees and general public who travel by bus.

- **Transport administrators / operators:** staff who manage vehicles, routes, stops and schedules.
- **Drivers (via location source):** whose vehicles supply the location updates.
- **Transport authorities and planners:** who may analyse historical data to improve services.

CHAPTER 2 - SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is lightweight and can be developed and run on an ordinary personal computer. The minimum and recommended configurations are given below.

Component	Minimum	Recommended
Processor	Intel Core i3 / equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB or more
Storage	10 GB free	20 GB SSD
Network	Broadband / Wi-Fi	Stable broadband connection
Display	1366 x 768	1920 x 1080
Location source (optional)	Smartphone browser GPS or simulator	GPS module (e.g. NEO-6M) on vehicle

2.2 Software Requirements

Category	Software
Operating System	Windows 10/11, Ubuntu or macOS
Frontend	React.js, HTML5, CSS3, JavaScript (ES6)
Backend	Node.js (LTS) and Express.js
Database	MySQL 8.x
Real-time	WebSocket (Socket.IO / ws)
Maps	Google Maps API or OpenStreetMap with Leaflet
Browser	Google Chrome, Microsoft Edge or Firefox (latest)

2.3 Development Tools and Frameworks

- **Visual Studio Code:** source code editor with extensions for JavaScript, React and SQL.
- **Git and GitHub:** version control and collaboration among team members.
- **Postman:** testing and documenting REST APIs.
- **MySQL Workbench:** designing the schema, ER diagrams and running queries.
- **npm:** package management for both frontend and backend.
- **Chrome DevTools:** debugging, network and performance inspection.

CHAPTER 3 - TECHNOLOGY STACK

3.1 Front-End

The user interface is built with **React.js**, whose component-based model suits a dashboard with many independently updating elements such as the map, vehicle list, route panel and notification area. HTML5 and CSS3 provide structure and responsive styling so that the application works on phones, tablets and desktops. JavaScript (ES6+) handles interaction, API calls and WebSocket events. The map is rendered with Leaflet and OpenStreetMap tiles (or the Google Maps JavaScript API).

3.2 Back-End

The backend uses **Node.js** with the **Express.js** framework. Node's non-blocking, event-driven architecture is well suited to handling many concurrent connections and a constant stream of small location messages. Express provides routing, middleware for authentication, validation and error handling, and a clean structure for REST controllers.

3.3 Database

MySQL is used as the relational database. The data in this project is highly structured and relational (routes contain ordered stops, vehicles are assigned to routes, schedules reference routes and stops), which makes a relational model the natural choice. MySQL offers transactions, foreign-key integrity, indexing and mature tooling. Location history is indexed on vehicle and timestamp to support fast retrieval of the latest position.

3.4 Version Control

The team uses **Git** with a shared **GitHub** repository. A main branch holds stable code while each member works on feature branches (for example `frontend-map`, `backend-api` and `database-schema`) that are merged through pull requests after review. This allowed three developers to work in parallel without overwriting each other's work and gave a complete history of changes.

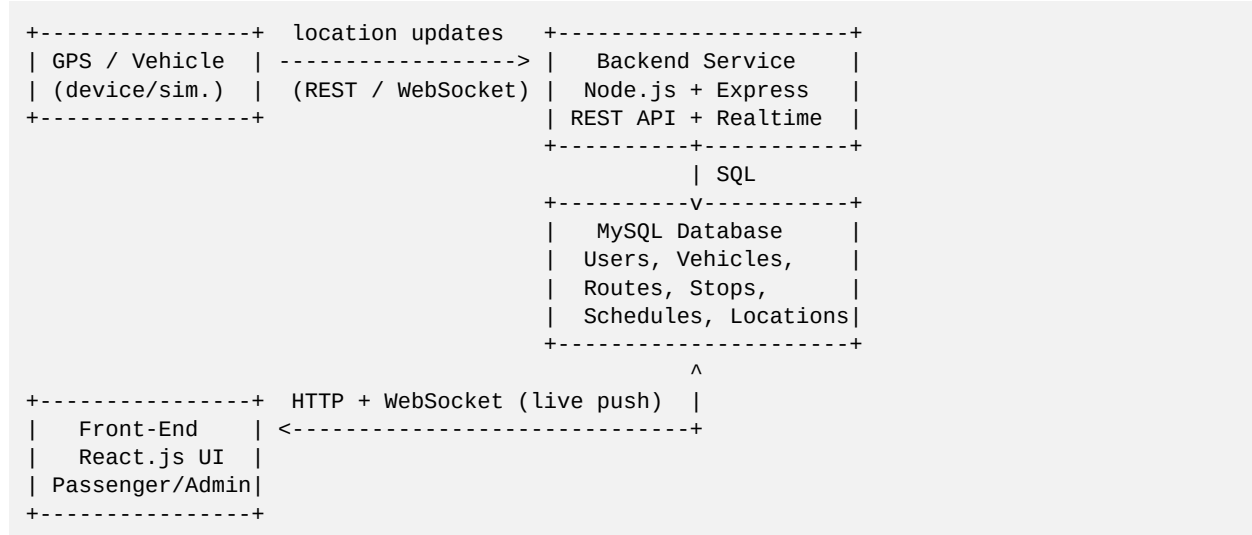
3.5 APIs or External Services

- **REST API:** request/response communication for routes, stops, vehicles, schedules, users and ETA queries.
- **WebSocket:** persistent two-way connection used to push live vehicle positions and notifications to connected clients.
- **Google Maps API / OpenStreetMap:** map tiles, markers and polylines for displaying vehicles and routes.
- **Browser Geolocation API:** optional source of location updates from a driver's smartphone.

CHAPTER 4 - SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a layered, client-server architecture. Vehicle location updates flow from the GPS source to the backend, are stored in the database, and are then pushed to the passengers' browsers.



4.2 Description of Each Layer (Frontend, Backend, Database)

Frontend layer. The React application presents the map, route list, stop details, ETA and notifications. It calls REST endpoints for static data (routes, stops, schedules) and subscribes to a WebSocket channel for live position updates, redrawing vehicle markers without a page reload.

Backend layer. The Express server receives location updates, validates them, writes them to the database and broadcasts them to subscribed clients. It also computes ETAs from the vehicle's last known position, remaining stops and recent average speed, enforces authentication for admin actions, and exposes the REST API.

Database layer. MySQL stores all persistent data. Normalized tables avoid redundancy, foreign keys preserve integrity, and indexes on (`vehicle_id`, `recorded_at`) make the latest-position query fast even as the history grows.

Separation of the three layers means each can be modified or scaled independently: the interface can be redesigned without changing backend logic, and the database can be moved to another server without affecting users.

4.3 Deployment Architecture

During development, all components run locally (React dev server, Node.js server and MySQL instance). For deployment, the React build is served as static files from a hosting service or a CDN, the Node.js backend runs on a cloud virtual machine or platform service behind a reverse proxy (Nginx) with HTTPS, and MySQL runs as a managed database or on the same server. GitHub Actions can be used as a simple CI/CD pipeline that runs tests and redeploys the application when code is merged to the main branch.

CHAPTER 5 - DESIGN

5.1 Use Case and Data Flow

The system has two principal actors. A **Passenger** can search routes, select a vehicle or stop, view live location and ETA, view schedules and receive notifications. An **Admin** can log in, manage vehicles, routes, stops and schedules, and publish service notices. The **GPS / Vehicle** acts as an external actor that sends location updates.

In the data flow, a location update enters the backend, is validated and stored, and is then broadcast to clients viewing that route. When a passenger selects a stop, the backend retrieves the vehicle's latest location and the remaining stops on its route and returns the computed ETA.

5.2 ER Diagram / Database Schema

The main entities and their relationships are: a Route has many Stops (through Route_Stops, which stores stop order); a Vehicle is assigned to one Route; a Vehicle produces many Location records; a Schedule links a Route, a Stop and a time; Users may be passengers or admins; Notifications are created by admins and linked to a Route.

Table	Key Attributes	Relationships
users	user_id (PK), name, email, password_hash, role	Creates notifications (admin)
routes	route_id (PK), route_name, source, destination	1:N vehicles, 1:N schedules, M:N stops
stops	stop_id (PK), stop_name, latitude, longitude	M:N routes via route_stops
route_stops	route_id (FK), stop_id (FK), stop_order	Resolves routes-stops M:N
vehicles	vehicle_id (PK), reg_number, capacity, status, route_id (FK)	N:1 route, 1:N locations
schedules	schedule_id (PK), route_id (FK), stop_id (FK), arrival_time	N:1 route, N:1 stop
locations	location_id (PK), vehicle_id (FK), latitude, longitude, speed, recorded_at	N:1 vehicle
notifications	notif_id (PK), route_id (FK), message, created_by (FK), created_at	N:1 route, N:1 user

Core tables are in Third Normal Form: non-key attributes depend only on the key, and the many-to-many relationship between routes and stops is resolved through the route_stops table.

```
CREATE TABLE locations (
  location_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  vehicle_id INT NOT NULL,
  latitude DECIMAL(9,6) NOT NULL,
  longitude DECIMAL(9,6) NOT NULL,
  speed DECIMAL(5,2),
  recorded_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),
  INDEX idx_vehicle_time (vehicle_id, recorded_at)
);
```

CHAPTER 6 - IMPLEMENTATION

6.1 Module-wise Implementation

- **Location Tracking Module:** receives GPS updates, validates coordinates, stores them and broadcasts them over WebSocket.
- **Route and Stop Module:** CRUD operations for routes and stops, and retrieval of ordered stops for a route.
- **Vehicle Module:** maintains vehicle details, status (active, idle, maintenance) and route assignment.
- **Schedule Module:** stores and serves scheduled arrival times for each stop.
- **ETA Module:** estimates arrival using the distance between the vehicle and the stop and the recent average speed.
- **Notification Module:** lets admins publish delay or service messages that are pushed to passengers.
- **User and Admin Module:** registration, login and role-based access.

6.2 Front-End Logic

The React application is organized into components such as MapView, RouteSelector, StopList, VehicleCard, EtaPanel, NotificationBell and AdminDashboard. State (selected route, vehicle positions, notifications) is held with React hooks (useState, useEffect) and shared through context. A WebSocket connection is opened in a useEffect hook; each incoming message updates the vehicle's coordinates in state, causing the marker on the map to move smoothly.

6.3 API Integration

Method	Endpoint	Purpose
GET	/api/routes	List all routes
GET	/api/routes/:id/stops	Ordered stops of a route
GET	/api/vehicles/:id/location	Latest position of a vehicle
GET	/api/stops/:id/eta	ETA of next vehicles at a stop
GET	/api/schedules?route=:id	Schedule for a route
POST	/api/locations	Vehicle sends a location update
POST	/api/auth/login	Login and receive JWT
POST	/api/notifications	Admin publishes a notice (protected)

```
// Receive a location update and broadcast it
app.post('/api/locations', async (req, res) => {
  const { vehicleId, latitude, longitude, speed } = req.body;
  await db.query(
    'INSERT INTO locations (vehicle_id, latitude, longitude, speed) VALUES (?, ?, ?, ?)',
    [vehicleId, latitude, longitude, speed]);
  io.to('route-' + routeOf(vehicleId)).emit('position',
    { vehicleId, latitude, longitude });
  res.status(201).json({ ok: true });
});
```

```
});
```

6.4 Authentication and Authorization

Admin users authenticate with email and password. Passwords are stored as bcrypt hashes, never in plain text. After successful login the server issues a JSON Web Token (JWT) that the client sends in the Authorization header. Express middleware verifies the token and checks the user's role before allowing access to protected endpoints such as creating routes or publishing notifications. Passenger-facing read endpoints remain public so that tracking requires no login. All queries use parameterized statements to prevent SQL injection.

CHAPTER 7 - FEATURES

7.1 List of Main Features

- Live vehicle tracking on an interactive map.
- Route information with ordered stops.
- Estimated arrival time at selected stops.
- Stop details including location and serving routes.
- Vehicle information (registration, capacity, status).
- Transport schedule display.
- Notifications and service status updates.
- Admin dashboard for managing data.
- Responsive interface for mobile and desktop.

7.2 How Each Feature Works

Feature	User perspective	Technical description
Live tracking	Select a route and watch buses move on the map.	Backend broadcasts each stored location over WebSocket; React updates marker positions.
Route and stops	See the full list of stops in order.	Joins routes, route_stops and stops ordered by stop_order.
ETA	See 'arriving in 6 min' for a chosen stop.	Distance from last location to stop divided by recent average speed, adjusted for intermediate stops.
Schedule	Check planned timings.	Query on schedules table filtered by route and stop.
Vehicle info	Tap a bus to see its number and status.	Reads vehicles table joined with latest location.
Notifications	Receive delay or service notices.	Admin POST stores notice and emits a WebSocket event to the route's room.
Admin dashboard	Add or edit vehicles, routes, stops, schedules.	JWT-protected CRUD endpoints with role checks.

CHAPTER 8 - TESTING

8.1 Postman Testing (Frontend/Backend)

Every REST endpoint was exercised in Postman using a shared collection. For each endpoint we verified the HTTP status code, response body structure, behaviour for missing or invalid input (400 responses), and access control (401/403 for protected endpoints without a valid token). Environment variables stored the base URL and JWT so the same collection could run against local and deployed servers.

8.2 Integration Testing

Integration tests confirmed that the layers work together end to end: a simulated vehicle posts a location, the record appears in MySQL, the WebSocket event reaches a connected browser, and the marker moves on the map. ETA results were checked against manually computed values for several vehicle positions.

8.3 Tools Used for Testing

- Postman for API testing.
- Jest and Supertest for backend unit and endpoint tests.
- A Node.js script that simulates vehicles moving along a route.
- Chrome DevTools for network and WebSocket inspection.
- MySQL Workbench for verifying stored data.

8.4 Test Cases and Results

ID	Test case	Expected result	Result
TC1	GET /api/routes	200 with list of routes	Pass
TC2	POST valid location update	201, row stored, event broadcast	Pass
TC3	POST location with invalid latitude	400 validation error	Pass
TC4	GET ETA for valid stop	200 with ETA in minutes	Pass
TC5	Admin login with correct credentials	200 and JWT returned	Pass
TC6	Admin login with wrong password	401 Unauthorized	Pass
TC7	Create route without token	401 Unauthorized	Pass
TC8	Passenger opens map, vehicle moves	Marker updates without refresh	Pass
TC9	Admin publishes notification	Passengers on route see notice	Pass

CHAPTER 9 - DEPLOYMENT

9.1 Steps to Deploy

- Clone the repository from GitHub on the server.

- Install Node.js and MySQL; create the database and run the schema script.
- Install backend dependencies with `npm install` and set environment variables.
- Build the React frontend with `npm run build`.
- Start the backend using a process manager such as PM2.
- Configure Nginx as a reverse proxy, serve the frontend build and enable HTTPS.
- Verify the deployment by opening the site and running the Postman collection against the live URL.

9.2 Environment Configuration

```
PORT=5000
DB_HOST=localhost
DB_USER=tracker_user
DB_PASSWORD=*****
DB_NAME=transport_tracker
JWT_SECRET=*****
MAP_API_KEY=*****
```

Secrets are kept in a `.env` file that is excluded from version control through `.gitignore`.

9.3 Hosting of Frontend and Backend

The React build can be hosted as static files on services such as Netlify, Vercel or GitHub Pages, or served by Nginx. The Node.js backend can be hosted on a cloud VM or on a platform such as Render or Railway, with MySQL running on the same server or as a managed database. CORS is configured so that only the frontend's origin may call the API.

CHAPTER 10 - CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Handling a continuous stream of location updates without overloading the database.
- Producing reasonably accurate ETAs when vehicle speed varies with traffic.
- Keeping the map smooth when many markers update frequently.
- Coordinating work among three team members and avoiding merge conflicts.
- Dropped WebSocket connections on unstable networks.
- Testing without real GPS hardware.

10.2 Solutions Applied

- Updates are validated and throttled, and an index on `(vehicle_id, recorded_at)` keeps queries fast.
- ETA uses a moving average of recent speeds rather than a single reading.
- The frontend updates only the changed marker and animates between positions.
- Feature branches and pull-request reviews on GitHub reduced conflicts.
- The client automatically reconnects and fetches the latest position on reconnect.
- A vehicle simulator script generated realistic movement along stored routes.

10.3 Current Limitations or Known Bugs

- ETA does not yet account for live traffic, signals or stoppage time at stops.
- Location data comes from a simulator or a smartphone rather than dedicated hardware.
- Notifications are shown in the application only; there are no SMS or push notifications.
- The system has been tested with a small number of simulated vehicles and has not been load-tested at city scale.

CHAPTER 11 - FUTURE ENHANCEMENTS

11.1 Planned Features

- Native Android and iOS applications for passengers and drivers.
- Push, SMS and email alerts for a chosen bus or stop.
- Crowd-level indicators showing how full a bus is.
- Multilingual interface.
- Journey planner combining multiple routes and transfers.
- Favourite routes and personalised alerts.

11.2 Possible Integrations or Optimizations

- Integration with live traffic data to improve ETA accuracy.
- Machine-learning models trained on historical location data for delay prediction.
- Redis caching of latest vehicle positions to reduce database reads.
- Time-series partitioning or archival of old location records.
- Horizontal scaling of the real-time service behind a load balancer.
- Integration with metro and ticketing systems.

CHAPTER 12 - CONCLUSION

12.1 Summary of the Project

Public Transport Live Tracker is a web-based system that lets passengers see where public transport vehicles are, which stops they will serve and when they are expected to arrive. A React frontend, a Node.js and Express backend with REST and WebSocket interfaces, and a normalized MySQL database work together to collect, store and deliver location data in real time.

12.2 What Was Achieved

- A normalized relational schema for users, vehicles, routes, stops, schedules, locations and notifications.

- A working backend with REST APIs, WebSocket broadcasting, JWT authentication and role-based access.
- An interactive map interface with live vehicle markers, route and stop details, schedules and ETA.
- An admin dashboard for managing transport data and publishing notices.
- Verified behaviour through Postman, integration and unit tests.

12.3 Skills Learned During Development

- Relational database design, normalization and indexing.
- Building REST APIs and real-time services with Node.js and Express.
- Component-based frontend development with React and map libraries.
- Securing applications with hashing, JWT and parameterized queries.
- Collaborative development with Git and GitHub, API testing and deployment.
- Teamwork, planning, documentation and presenting technical work.

CHAPTER 13 - REFERENCES

- Node.js Documentation - <https://nodejs.org/en/docs>
- Express.js Documentation - <https://expressjs.com>
- React Documentation - <https://react.dev>
- MySQL 8.0 Reference Manual - <https://dev.mysql.com/doc>
- Socket.IO Documentation - <https://socket.io/docs>
- Leaflet Documentation - <https://leafletjs.com>
- OpenStreetMap - <https://www.openstreetmap.org>
- Google Maps Platform Documentation - <https://developers.google.com/maps>
- Postman Learning Center - <https://learning.postman.com>
- JSON Web Tokens - <https://jwt.io>
- Elmasri, R. and Navathe, S., *Fundamentals of Database Systems*, Pearson.
- Silberschatz, A., Korth, H. and Sudarshan, S., *Database System Concepts*, McGraw-Hill.

CHAPTER 14 - APPENDICES

A. Screenshots of the Application

Screenshots of the home page, live map, route and stop details, ETA panel and admin dashboard are to be inserted here.

B. Sample Code Snippets

Representative code for the location API and database schema is given in Chapters 5 and 6.

C. Installation / Setup Instructions

- Install Node.js (LTS) and MySQL 8.
- Clone the repository and create the database using the provided SQL script.
- Run `npm install` in the backend and frontend folders.
- Create the `.env` file with database and JWT settings.
- Start the backend with `npm start` and the frontend with `npm run dev`.
- Open the application in a browser.

D. User Manual

Passengers: open the site, choose a route, tap a vehicle or stop to see live position, schedule and ETA, and check the notification bell for service updates. **Admins:** log in, then use the dashboard to add or edit vehicles, routes, stops and schedules, and to publish notices.

E. Geo-Tag Photos with Guide

To be attached.

F. Review Forms with Guide Signatures

To be attached.